Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	02-03-2026		

Experiment 5

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

1 Aim and Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA)
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions

Students must select, tune, and justify hyperparameters such as learning rate, activation function, optimizer, batch size, and number of hidden layers.

2 Dataset Description

English Handwritten Characters Dataset

Download Link: https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset

- Total samples: 3,410
- Number of classes: 62 (0–9, A–Z, a–z)
- Image type: Grayscale

3 Dataset Description and preprocessing

```
# Import libraries
import numpy as np
import pandas as pd
import os
import cv2
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report, roc_curve, auc
from sklearn.preprocessing import label_binarize

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load CSV file
df = pd.read_csv(r"Dataset/english.csv")
print(df.head())

image_size = 28   # resize to 28x28
data = []
labels = []
base_path = "Dataset/Img"
for i in range(len(df)):
    # Get image path and label
    img_path = df.iloc[i]['image']   # Img/img001-001.png
    label = df.iloc[i]['label']
    # Extract only filename from CSV path
    img_name = os.path.basename(img_name)
    # Create correct full path
    img_path = os.path.join(base_path, img_name)
    try:
        # Read grayscale image
        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        if image is None:
            continue
        # Resize
        image = cv2.resize(image, (image_size, image_size))
        # Normalize
        image = image / 255.0
        # Flatten
        image = image.flatten()
        data.append(image)
        labels.append(label)
    except:
        pass
X = np.array(data)
y = np.array(labels)
print("Dataset shape:", X.shape)
print("Labels shape:", y.shape)

# Encoding
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
num_classes = len(np.unique(y_encoded))
print("Number of classes:", num_classes)

# Splitting the data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42
)
```

4 PLA Implementation and Results

```
# Step Activation function
def step_function(x):
    return np.where(x >= 0, 1, 0)

# Single perceptron class
class Perceptron:
    def __init__(self, lr=0.01, epochs=50):
        self.lr = lr
        self.epochs = epochs
    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        for _ in range(self.epochs):
            for i in range(n_samples):
                linear = np.dot(X[i], self.weights) + self.bias
                y_pred = step_function(linear)

                update = self.lr * (y[i] - y_pred)
                self.weights += update * X[i]
                self.bias += update
    def predict(self, X):
        linear = np.dot(X, self.weights) + self.bias
        return step_function(linear)

# Multi Class PLA using One-vs-Rest
class MultiClassPLA:
    def __init__(self, lr=0.01, epochs=50):
        self.lr = lr
        self.epochs = epochs
        self.models = []
    def fit(self, X, y, n_classes):
        self.models = []
        for i in range(n_classes):
            binary_y = np.where(y == i, 1, 0)
            model = Perceptron(self.lr, self.epochs)
            model.fit(X, binary_y)
            self.models.append(model)
    def predict(self, X):
        outputs = []
        for model in self.models:
            linear = np.dot(X, model.weights) + model.bias
            outputs.append(linear)

        outputs = np.array(outputs)
        return np.argmax(outputs, axis=0)

# Train PLA
print("Training PLA...")
pla = MultiClassPLA(lr=0.01, epochs=50)
pla.fit(X_train, y_train, num_classes)
pla_pred = pla.predict(X_test)

pla_pred_train = pla.predict(X_train)
# Misclassification loss
final_train_loss_pla = np.mean(pla_pred_train != y_train)
```

```
print("Final Training Loss (PLA - Error Rate):", final_train_loss_pla)

# Evalauate PLA
print("PLA Results")
print("Accuracy:", accuracy_score(y_test, pla_pred)*100,  "%")
print("Precision:", precision_score(y_test, pla_pred, average='weighted'))
print("Recall:", recall_score(y_test, pla_pred, average='weighted'))
print("F1 Score:", f1_score(y_test, pla_pred, average='weighted'))
```

5 MLP Implementation and Results

```
# Implement Multilayer Perceptron (MLP)
#Convert Labels to One-Hot
y_train_cat = tf.keras.utils.to_categorical(y_train, num_classes)
y_test_cat = tf.keras.utils.to_categorical(y_test, num_classes)

# Multi Class PLA using One-vs-Rest
class MultiClassPLA:
    def __init__(self, lr=0.01, epochs=50):
        self.lr = lr
        self.epochs = epochs
        self.models = []

    def fit(self, X, y, n_classes):
        self.models = []
        self.loss_history = []

        for i in range(n_classes):
            binary_y = np.where(y == i, 1, 0)

            model = Perceptron(self.lr, self.epochs)
            model.fit(X, binary_y)
            self.models.append(model)

            # ---- compute training error after each epoch ----
            pred_all = self.predict(X)
            loss = np.mean(pred_all != y)   # misclassification loss
            self.loss_history.append(loss)

    def predict(self, X):
        outputs = []

        for model in self.models:
            linear = np.dot(X, model.weights) + model.bias
            outputs.append(linear)

        outputs = np.array(outputs)
        return np.argmax(outputs, axis=0)

    def decision_function(self, X):
        return np.dot(X, self.weights.T) + self.bias

# Train MLP
print("Training MLP...")

mlp = build_mlp(hidden_layers=[256,128],
```

```
                 activation='relu',
                 optimizer='adam',
                 lr=0.001)

history = mlp.fit(
    X_train, y_train_cat,
    epochs=20,
    batch_size=64,
    validation_split=0.1
)

# Final Training Loss (MLP)
final_train_loss_mlp = history.history['loss'][-1]
print("Final Training Loss (MLP):", final_train_loss_mlp)

# Evaluate MLP
mlp_pred = np.argmax(mlp.predict(X_test), axis=1)

print("MLP Results")

print("Accuracy:", accuracy_score(y_test, mlp_pred)*100, "%")
print("Precision:", precision_score(y_test, mlp_pred, average='weighted'))
print("Recall:", recall_score(y_test, mlp_pred, average='weighted'))
print("F1 Score:", f1_score(y_test, mlp_pred, average='weighted'))
```

6 Hyperparameter Tuning Analysis

```
# Hypertune
configs = [
    ([128], 'relu', 'sgd', 0.01, 32),
    ([256,128], 'relu', 'adam', 0.001, 64),
    ([512,256,128], 'tanh', 'adam', 0.0005, 64)
]

result = []

for layers, act, opt, lr, batch in configs:
    print("\nTraining config:", layers, act, opt, lr)

    model = build_mlp(layers, act, opt, lr)

    model.fit(X_train, y_train_cat, epochs=10, batch_size=batch, verbose=0)

    pred = np.argmax(model.predict(X_test), axis=1)
    acc = accuracy_score(y_test, pred)

    result.append((layers, act, opt, lr, batch, acc))

print("\nHyperparameter Results")
Tune_Results = pd.DataFrame({
    "Hidden Layers": [result[0][0], result[1][0], result[2][0]],
    "Activation": [result[0][1], result[1][1], result[2][1]],
    "Optimizer": [result[0][2], result[1][2], result[2][2]],
    "Learning Rate": [result[0][3], result[1][3], result[2][3]],
    "Batch Size": [result[0][4], result[1][4], result[2][4]],
    "Accuracy": [result[0][5]*100, result[1][5]*100, result[2][5]]*100
```

```
})
Tune_Results

print("\nFinal Comparison")
print("PLA Accuracy:", accuracy_score(y_test, pla_pred))
print("MLP Accuracy:", accuracy_score(y_test, mlp_pred))
```

7 Visualizations

This section presents the visual analysis of the confusion matrix, ROC Curves and Training loss vs Epochs curve

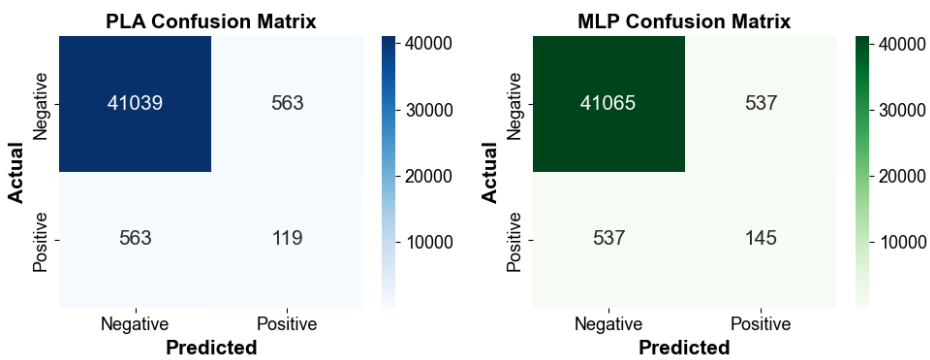

Figure 1: Confusion Matrix

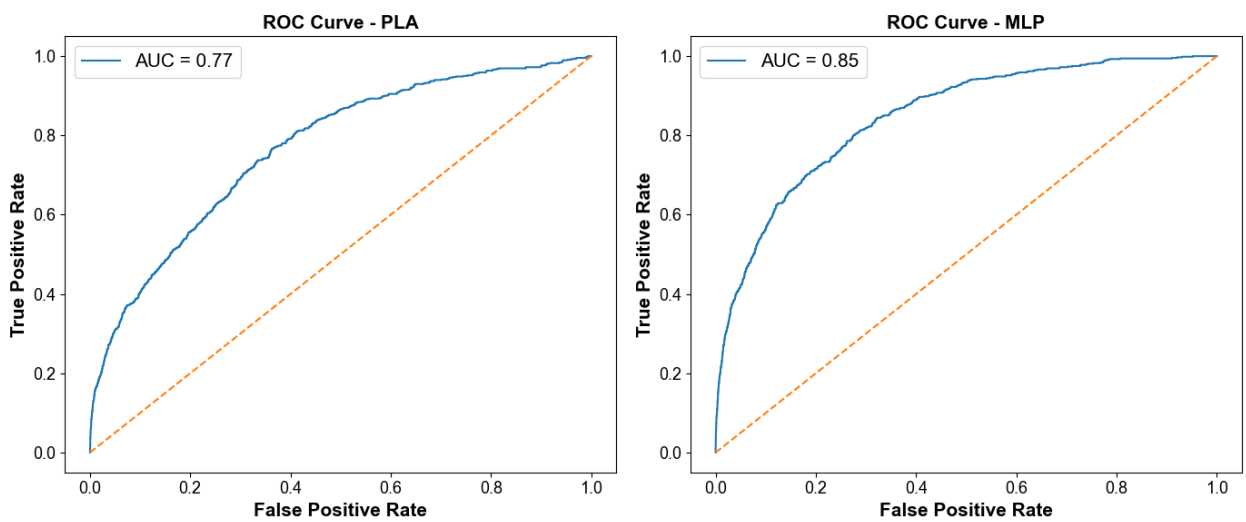

Figure 2: ROC Curves (micro/macro average)

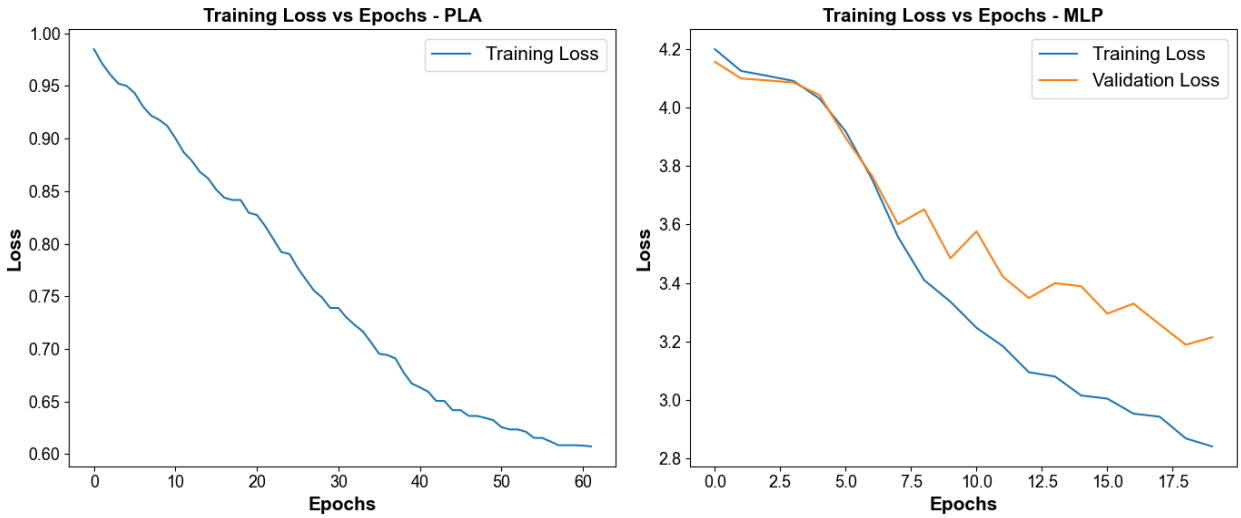

Figure 3: Training loss vs epochs curve

8 Performance Comparison Tables

Table 1: Overall Performance Comparison between PLA and MLP

Model	Accuracy (%)	Precision	Recall	F1-score
PLA (OvR)	17.45	0.27	0.17	0.15
MLP (Tuned)	21.26	0.23	0.21	0.18

Table 2: Hyperparameter Tuning Results for MLP

Hidden Layers	Activation	Optimizer	Learning Rate	Batch Size	Accuracy (%)
(128)	ReLU	SGD	0.01	32	4.98
(256–128)	ReLU	Adam	0.001	64	2.78
(512–256–128)	Tanh	Adam	0.0005	64	21.55

Training Convergence Comparison

Table 3: Kernel-wise Performance Comparison of SVM

Model	**Epochs**	**Final Training Loss**	**Convergence Behavior**
PLA	50	0.61	Non-convergent
MLP (Tuned)	20	2.84	Stable convergence

9 Observations and Conclusion

- **Why does PLA underperform compared to MLP?**
  Perceptron Learning Algorithm (PLA) is a simple linear classifier that can only learn linearly separable patterns. It cannot capture complex non-linear relationships in image data. In contrast, Multi-Layer Perceptron (MLP) contains hidden layers and non-linear activation functions, which enable it to learn complex feature representations and improve classification performance.

- **Which hyperparameter had the most impact on MLP performance?**
  The number of hidden layers and neurons had the most significant impact, as they determine the model's capacity to learn complex patterns. The learning rate also strongly influenced performance by affecting how quickly and effectively the model converges. Additionally, the optimizer choice impacted training efficiency and final accuracy.

- **How does optimizer choice affect convergence?**
  The optimizer controls how model weights are updated during training. Optimizers like Adam generally converge faster and more efficiently due to adaptive learning rates, while SGD may converge more slowly and can get stuck in local minima if not properly tuned. Thus, optimizer selection directly affects training speed, stability, and final model performance.

- **Does increasing hidden layers always improve performance?**
  No, increasing hidden layers does not always improve performance. While deeper networks can learn more complex features, excessive layers may lead to overfitting, increased computational cost, and training difficulties such as vanishing gradients. Proper tuning and validation are required to determine the optimal network depth.

- **How can overfitting be detected and mitigated?**
  Overfitting can be detected when training accuracy is very high but validation accuracy is low, or when validation loss increases while training loss decreases. It can be mitigated using techniques such as regularization, dropout, early stopping, data augmentation, and reducing model complexity.